

COMS 4444 Project 1 Guide

The problem

n people share one drawer of C socks, half white, half black. Each day, in random order, each person draws 4 socks (or 5), wears two, and puts the rest back or throws them away. Worn socks fade. A pair whose shades differ by **more than 6** costs $|s_1 - s_2|$ embarrassment; a difference of exactly 6 is free. Worn-out socks (white 127, black 64) have a 25% chance of a hole and are thrown out. After six discards of one colour, Bob buys a \$10 six-pack if the budget covers it. If money runs out, the drawer only shrinks. If fewer than two socks are offered, that counts as sockless, with a penalty of **65536**.

You only write **which two to wear and what to throw away**. The engine does everything else.

Where to write code

You edit **one file**. Nothing else.

```
cp -r players/player_template players/player_4 # use your group number
```

Then in `players/player_4/player.py`:

1. Rename `PlayerTemplate` to `Player4`
2. Keep `__init__.py` in that folder
3. Fill in `select_socks`

```
def select_socks(self, offered, turn) -> Selection:
    # offered = (255, 0, 253, 2) shades only, this handful only
    return Selection(wear=(0, 2), discard=()) # indices into offered
```

`wear` = two distinct indices. `discard` = any leftover indices, or empty. Anything you neither wear nor discard goes back **unaged**. Do not edit `core/`, `ui/`, `models/`, or other groups' folders.

The 4 above is a placeholder. Wherever you see 4 in this guide, in the folder name, the class name, and the `--player` flag, replace it with your own assigned group number.

What you can see

Thing	Meaning
<code>offered[i]</code>	shade 0–255 of sock i in this handful
<code>turn.day</code>	today (1-based)
<code>turn.total_spent</code>	household dollars so far
<code>turn.budget_remaining</code>	money left (inf if unlimited)
<code>turn.embarrassment_history</code>	your daily scores
<code>self.capacity</code> , <code>self.roommates</code> , <code>self.selection_unit</code> , <code>self.days</code>	game parameters

Shade > 64 is white (255 to 127). Shade ≤ 64 is black (0 to 64). Inferring colour from shade is allowed.

What you cannot see

The rest of the drawer, other people's choices, discard counts, or sock identity across days. Index 0 tomorrow is a different sock.

If your code crashes, hangs, or returns junk, you **forfeit that turn**: the engine wears socks 0 and 1, discards nothing. Everyone else still plays.

How to run

Install `uv`, then from the repo:

```
uv run main.py # 4 random players, 360 days, JSON
uv run main.py --gui --player g 4 --days 360 --seed 7 --budget 2500
uv run main.py --player 4 2 --player g 2 # your group vs greedy
uv run main.py --unit 5 -C 44 --player 4 4 # five-sock variant
uv run main.py --player 4 4 --days 360 --budget 2500
```

Flag	Default	Meaning
--player CODE COUNT	r 4	CODE is r for random, g for greedy, or your group's number
-C	40	drawer size (multiple of 4, $> \text{unit} \times n + 10$)
--unit	4	socks drawn per person (4 or 5)
--days	360	one course year
--seed	4444	same seed gives same run
--budget	unlimited	dollars for the whole run
--gui	off	visualizer

You choose days/budget **on your laptop**. Class demos and the tournament use whatever the TA sets (tournament is currently **1080 days, \$7490**).

What the window shows

```
uv run main.py --gui --player g 4 --days 360 --seed 7 --budget 2500
```

On screen	What it is
Day N / D	simulated morning
\$ spent / budget bar	household money. When it hits \$0, nothing is replaced again
Drawer held of C	socks still in the shared drawer (true grey = true shade)
Trash	thrown away, waiting for a 6-pack (6 of one colour buys \$10 new socks)
Roommate row	that person's handful this turn , not the whole drawer

On a roommate row: **wear** (green) is the two they put on, **toss** (red) is what they threw away, **back** is a leftover returned with its shade unchanged, **wore** 255 / 255 **diff** 0 is today's pair and $|\text{s1}-\text{s2}|$, **today** / **total** is embarrassment today and so far, and **#1 #2** is the order they dressed that day (reshuffled every morning).

Keys: RIGHT next roommate (after the last, next day). LEFT previous roommate, **same day only**. SPACE play (whole days). UP/DOWN speed. Q quit. A red **FAULT** banner means that group's code failed; they forfeited the turn.

Greedy (g) is a baseline, not the answer. It wears the closest pair and may toss a leftover that is > 6 shades from that pair, including a brand-new opposite-colour sock, but only while `turn.budget.remaining` still covers a \$10 pack. With an unlimited budget that value is `inf`, so greedy keeps tossing forever. Beat it.

The three goals

These are sock simulations you watch with `--gui` and then report on. Swap `g` for your group number once your player exists. SPACE plays the year; Q quits.

Goal 1: given a budget, stay under it and keep embarrassment down. One course year, \$2500 for the household (roughly the tournament rate of \$7490 over 1080 days).

```
uv run main.py --gui --player 4 4 --days 360 --seed 7 --budget 2500
```

Report spend against \$2500, embarrassment per day, and sockless days. Holes still cost money even if you never toss extra socks.

Goal 2: four socks versus five, same budget. Same year and \$2500, but each person is handed 5. `C=44` because the drawer must be bigger than $5 \times 4 + 10$.

```
uv run main.py --gui --unit 5 -C 44 --player 4 4 --days 360 --seed 7 --budget 2500
```

Report whether embarrassment dropped compared to Goal 1, and by how much. Same seed, so the comparison is fair.

Goal 3: pooled drawer versus each person alone. Same socks per person: 4 people sharing 64 versus 1 person with 16. A solo player can track the whole drawer; that is allowed.

```
uv run main.py --gui --player 4 4 -C 64 --days 360 --seed 7 --budget 2500
uv run main.py --gui --player 4 1 -C 16 --days 360 --seed 7 --budget 625
```

\$625 is \$2500 divided by 4, so the solo run has the same money per person. Report which setting has lower embarrassment.

To average out noise, drop `--gui` and add more seeds, or run `sweep.py` on `configs/goal1_budget_sweep.json`, `goal2_unit_comparison.json`, or `goal3_pooling.json`.

Test your player

```
# it loads
uv run main.py --player 4 1 --player g 3 --days 30 --seed 1

# watch it (Goal 1)
uv run main.py --gui --player 4 4 --days 360 --seed 7 --budget 2500

# tournament length
uv run main.py --player 4 4 --days 1080 --budget 7490 --seed 7
```

Startup warnings mean your folder or class didn't load (`__init__.py` missing, not named `player_4`, class not `Player4`).